

class030 异或运算的骚操作

本站视频: [算法讲解030【必备】异或运算的骚操作](#)
 前置知识: class003 二进制和位运算

核心知识：异或运算的本质 = 无进位相加

不要记"不同为1、相同为0"，会和同或运算搞混。用**无进位相加**理解：

```

1 |   A = 1 0 0 1 1
2 |   B = 0 1 1 0 1
3 | A⊕B = 1 1 1 1 0   ← 每位独立相加，丢弃所有进位

```

四大性质（全由无进位相加推导）

性质	描述	用途
① 无进位相加	每位独立求和，偶数个1消为0，奇数个1剩1	根本性质，其余都由此推出
② 交换律+结合律	同一批数字，不管什么顺序异或，结果都一样	乱序等价，分组自由
③ $N \oplus N = 0, 0 \oplus N = N$	自己异或自己得0，0异或任何数得原数	偶数次消掉，奇数次留下
④ 部分=整体⊕另一部分	若 $A \oplus B = C$ ，则 $A = C \oplus B, B = C \oplus A$	已知整体和一部分，可求另一部分

性质②的证明（关键理解）

同一批数字异或结果不变，因为结果**只和每一位上1的个数有关**：

```

1 | 统计每一位上一共多少个1：
2 | - 偶数个1 → 无进位相加 → 0（全消掉）
3 | - 奇数个1 → 无进位相加 → 1（剩一个）
4 |
5 | 顺序怎么换，每位上1的总数不变 → 结果一定一样

```

引子：黑白球问题

问题：袋中有 A 个白球、B 个黑球。每次随机取两球：

- 两球同色 → 放回一个**白**球
- 两球异色 → 放回一个**黑**球

最终只剩一球，请问是黑球的概率？（用 A、B 表示）

答案：与 A 无关！**B 为偶数** → 概率 **0%（必白）**；**B 为奇数** → 概率 **100%（必黑）**

解释：白=0，黑=1。整个过程等价于：每次取两个数，把它们的异或结果放回去。最终结果 = $A \text{ 个 } 0 \oplus B \text{ 个 } 1$

的整体异或 $\rightarrow$ 只取决于 B 个 1 的奇偶性。

题目1：异或交换两个数

技巧： 三行异或完成交换，不需要临时变量

```
1 a = a ^ b;  
2 b = a ^ b;  
3 a = a ^ b;
```

推导过程：

```
1 初始: a=甲, b=乙  
2 第一行后: a=甲⊕乙, b=乙  
3 第二行后: a=甲⊕乙, b=甲⊕乙⊕乙=甲⊕0=甲  
4 第三行后: a=甲⊕乙⊕甲=乙, b=甲
```

⚠ **坑点：** 数组中 `swap(arr[i], arr[j])` 必须保证 `i != j`

当 `i==j` 时，同一个内存位置做三次异或：

```
1 arr[0] = arr[0] ^ arr[0] = 0  ← 第一次就把自己刷成0了!  
2 arr[0] = arr[0] ^ arr[0] = 0  
3 arr[0] = arr[0] ^ arr[0] = 0
```

所以这种写法知道即可，不推荐使用。

题目2：不用任何判断和比较，返回两数最大值

测试链接：[牛客网 - 求最大最小值](#)

辅助函数

```
1 // 0变1, 1变0 (必须保证n是0或1)  
2 int flip(int n) { return n ^ 1; }  
3  
4 // 非负数返回1, 负数返回0  
5 int sign(int n) { return flip((unsigned int)n >> 31); }
```

sign 原理： 无符号右移31位，符号位到最低位。负数（最高位=1） $\rightarrow$ 移完得1；非负（最高位=0） $\rightarrow$ 移完得0。再 flip 一次得到"非负=1，负=0"。

版本1：有溢出风险

```
1 int getMax1(int a, int b) {
2     int c = a - b;           // 差值可能溢出!
3     int returnA = sign(c);    // c非负→1, c负→0
4     int returnB = flip(returnA);
5     return a * returnA + b * returnB;
6 }
```

思想： `returnA` 和 `returnB` 互斥（一个1一个0）， $ax1 + bx0 = a$ ， $ax0 + bx1 = b$ ，等价于 if-else 但没有写判断语句。

问题： `a - b` 可能溢出（如 `a=INT_MIN`，`b=INT_MAX`），溢出后符号位不可靠。

版本2：完全安全版

```
1 int getMax2(int a, int b) {
2     int c = a - b;
3     int sa = sign(a);    // a的符号
4     int sb = sign(b);    // b的符号
5     int sc = sign(c);    // c的符号
6
7     int diffAB = sa ^ sb;    // a和b符号不同→1, 相同→0
8     int sameAB = flip(diffAB); // a和b符号相同→1, 不同→0
9
10    // returnA由两种情况:
11    // 情况1: a、b符号不同 且 a非负 → 返回a（无需看c）
12    // 情况2: a、b符号相同 且 c非负 → 返回a（c不会溢出）
13    int returnA = diffAB * sa + sameAB * sc;
14    int returnB = flip(returnA);
15    return a * returnA + b * returnB;
16 }
```

为什么安全？ a、b同号时 a-b 绝不溢出；a、b异号时直接用 a 的符号判断（a非负 → a大）。两种情况互斥，不用 c 的符号去判断异号的情况。

题目3：找到缺失的数字 — LeetCode 268

测试链接：[LeetCode 268. Missing Number](https://leetcode.com/problems/missing-number/)

题意： 数组长度 `n`，包含 `[0, n]` 共 `n+1` 个数中除了某一个之外的所有数，乱序，找缺失的数。

思路：

```
1 eorAll = 0⊕1⊕2⊕...⊕n      ← 完整的所有数
2 eorHas = 数组中所有数的异或 ← 实际出现的数
3 缺失的数 = eorAll ⊕ eorHas  ← 利用性质⑥
```

代码：

```

1  int missingNumber(vector<int>& nums) {
2      int eorAll = 0, eorHas = 0;
3      for (int i = 0; i < (int)nums.size(); i++) {
4          eorAll ^= i;          // 异或上0~n-1
5          eorHas ^= nums[i];    // 异或上数组中的数
6      }
7      eorAll ^= nums.size();    // 最后异或上n
8      return eorAll ^ eorHas;
9  }

```

题目4：找唯一出现奇数次的数 — LeetCode 136

测试链接：[LeetCode 136. Single Number](https://leetcode.com/problems/single-number/)

题意：数组中只有一种数出现奇数次，其余都出现偶数次，返回该数。

LeetCode 阉割版：某个元素只出现一次，其余出现两次。**但实际方法更强**——只要一种出现奇数次、其余都出现偶数次（偶数次的次数可以各自不同），都成立。

思路：

```

1  全部异或到一起：
2  - 偶数次的数 → 自己数 ⊕ 自己数 = 0 → 消掉
3  - 奇数次的数 → 偶数份消掉，剩最后一个自己 → 留下

```

代码（一行核心）：

```

1  int singleNumber(vector<int>& nums) {
2      int eor = 0;
3      for (int num : nums) eor ^= num;
4      return eor;
5  }

```

时间复杂度 $O(n)$ ，额外空间 $O(1)$ 。

题目5：找唯二出现奇数次的数 — LeetCode 260

测试链接：[LeetCode 260. Single Number III](https://leetcode.com/problems/single-number-iii/)

题意：数组中有**两种**数出现奇数次（假设为 a 、 b ， $a \neq b$ ），其余数出现偶数次，找出 a 和 b 。

前置：Brian Kernighan 算法 — 提取最右侧的1

```

1  N & (-N) → 取出二进制中最右侧的那个1所在的状态
2
3  例：N = 0b01011000
4      -N = 0b10101000 （取反+1）
5      N & (-N) = 0b00001000 ← 只有最右侧的1保留

```

注意 C++ 中要处理 `INT_MIN` (-N 会溢出)，用 `unsigned int` 转换。

核心思路

1. `eor1 = 全部异或 = a ⊕ b` ($a \neq b \rightarrow$ eor1 必在某位上是1)
2. 用 Brian Kernighan 提取 `eor1` 最右侧的1，记作 `rightOne`
3. `rightOne` 上为1的那一位，a 和 b 一定不同 → **按这一位是否为1可以分成两组**
4. 用 `eor2` 只异或该位为0的那组数，得到 a 或 b 之一
5. 另一个 = `eor1 ⊕ eor2`

代码

```
1  vector<int> singleNumber(vector<int>& nums) {
2      int eor1 = 0;
3      for (int num : nums) eor1 ^= num;
4      // eor1 = a ^ b
5
6      // 提取最右侧的1 (C++注意INT_MIN)
7      unsigned int rightOne = (unsigned int)eor1 & (-(unsigned int)eor1);
8
9      int eor2 = 0;
10     for (int num : nums) {
11         if (((unsigned int)num & rightOne) == 0) {
12             eor2 ^= num;    // 只异或该位为0的数
13         }
14     }
15     // eor2 是 a,b 之一, 另一个 = eor1 ^ eor2
16     return {eor2, eor1 ^ eor2};
17 }
```

题目6：找唯一出现次数少于m的数 — LeetCode 137

测试链接：[LeetCode 137. Single Number II](https://leetcode.com/problems/single-number-ii/)

题意：数组中只有一种数出现次数小于 m，其余都恰好出现 m 次。返回那个不足 m 次的数。

LeetCode 特例： m=3，即一种数出现1次，其余出现3次。

核心思路（多位多进制）

把每个数看作 32 位二进制。统计**每一位上1的总个数**：

- 出现了 m 次的数，在它的每个为1的位上贡献恰好 m 个1 → **模 m 余 0**
- 那个不足 m 次的数，在它每个为1的位上贡献 k ($k < m$) 个1 → **模 m 余 k ≠ 0**

最后检查每位：`cnts[i] % m != 0` → 该位是1，设入答案。

代码

```
1  int find(vector<int>& arr, int m) {
2      int cnts[32] = {0};
3      for (int num : arr) {
4          for (int i = 0; i < 32; i++) {
5              cnts[i] += (num >> i) & 1;  // 统计第i位1的个数
6          }
7      }
8      int ans = 0;
9      for (int i = 0; i < 32; i++) {
10         if (cnts[i] % m != 0) {
11             ans |= (1 << i);  // 该位是1, 设入答案
12         }
13     }
14     return ans;
15 }
```

时间复杂度 $O(32n) = O(n)$, 额外空间 $O(1)$ 。

总结

题目	核心技巧	LeetCode
两数交换	三行异或, 必须 $i \neq j$	-
不用判断求最大值	<code>sign()</code> + 互斥 <code>returnA/returnB</code> , 分同号异号防溢出	牛客
缺失数字	完整 $\oplus$ 实际 = 缺失	268
一种数奇数次	全异或, 偶数消掉奇数留	136
两种数奇数次	Brian Kernighan 分组	260
一种数不足m次	32位统计, 模m不为0则设位	137

异或应用模式:

1. **消去偶次**: 偶数个相同的数异或得0
2. **分组**: 利用某位是否为1, 自然分为两组
3. **提取差异位**: Brian Kernighan 提取最右侧的1
4. **位统计+取模**: 解决"次数不等"类问题